Wajeh at AISA-ArabicFC: A Non-LLM Classical ML and Retrieval Pipeline for Arabic Function Calling

Abdullah Rashed Aldosari **Nader Hajed Shehri**
University of Bisha, Saudi Arabia
444800030@ub.edu.sa 443801387@ub.edu.sa

Abstract

We describe **NABIQ**, the system submitted by team Wajeh (واجه) to Tracks A and B of the AISA-ArabicFC shared task on Arabic function calling (Nacar et al., 2026a). NABIQ contains no large language model, no neural network of any kind, no external API and no GPU. It is a CPU-based pipeline of classical ML, retrieval and rule-based repair: a TF-IDF router, a per-tool nearest-argument retriever over train, span extractors with train-mined gazetteers, and a no-call detector. Its core mechanism is an *evidence gate*: a candidate value may replace a prediction only if it is a normalised substring of the utterance, a member of a closed canonical map, or a date whose tokens occur in the text. On the blind test the frozen system scored Overall A 0.7904 (rank 16) and Overall B 0.8282 (rank 15) at ThinkRate 1.0000. We keep five evaluation levels distinct because the submitted system drops 0.21 ArgEM from development to test. A rehearsal on 545 held-out training rows, run before the blind release, tracked our official Overall A far better than the tuned development estimate (composite error 0.0033 against 0.1724, cancelling larger FnAcc and ArgEM errors); that contrast, not the components, is our result.

1 Introduction

Function calling is the interface at which a language model stops describing an action and performs one. AISA-ArabicFC (Nacar et al., 2026a) is the first shared task to benchmark this capability in Arabic: given an utterance in one of five dialects (MSA, Gulf, Egyptian, Levantine, Maghrebi) and a candidate set from a 27-tool registry over eight service domains, a system must decide whether a call is required, select the tool, and emit its arguments as JSON. Track B adds an Arabic reasoning trace. Arguments are the harder sub-task: on the shared task's final evaluator the strongest published baselines reach argument exact match (ArgEM) of 0.744 (GPT-4o, zero-shot) and 0.679 (a fine-tuned 270M model, AISA-AR-FunctionCall-Think), each well below its own function-selection accuracy (Nacar et al., 2026a,b).

Our entry was deliberately contrarian. Instead of fine-tuning a generative model, we asked how far a non-neural, rule-governed system can go when, in the layers where the gate applies, a predicted value must satisfy explicit textual or canonical evidence criteria. NABIQ uses no LLM, no inference API, no PyTorch and no GPU; its only learned components are a TF-IDF representation and a logistic-regression classifier from `scikit-learn` (Pedregosa et al., 2011). It runs locally on CPU and is auditable line by line. That is what the evaluation and governance layers of the AISA reference architecture (Nacar et al., 2026c) ask of an agent whose calls must be explained after the fact.

We report three findings. First, evidence-gated extraction is competitive with learned decoding on argument accuracy: our blind-test ArgEM of 0.7081, from a system with no neural model, is on a par with a fine-tuned 270M model (0.679) and few-shot GPT-4o (0.699) on the final evaluator, below zero-shot GPT-4o (0.744); on Track B our perfect think coverage lifts us above the fine-tuned baseline (0.8282 against 0.8074). Second, the negative class is where routing performance breaks down. Our development FnAcc reached 1.0000, although that estimate was affected by the `negative_category` proxy in the no-call detector as well as by later rule-based no-call handling, and test FnAcc was 0.9138. Third, a zero-regression acceptance protocol is a weaker guarantee than it appears: 21 sequential accepts on the same 500 examples yielded a system that ranked high on development and 16th on the blind set, and a rehearsal on held-out training rows anticipated that outcome. We also report three engineering observations: nondeterminism from unordered iteration, an acceptance suite whose coverage did not

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 1564–1573
October 28-29, 2026 ©2026 Association for Computational Linguistics

reach the rule that later failed the validator, and a label proxy in the input that reaches the no-call detector's feature string. Code and the frozen submission package: https://github.com/tgfdru/nabiq-arabicfc.

2 Background

Task setup. Each instance provides a developer message carrying the candidate tool schemas and an Arabic user message. A Track A prediction is a JSON line with `id`, `tool_called` (a tool name or `"none"`) and `arguments`; Track B adds `think`. For كم تبلغ الجمارك على حقيبة يد قيمتها 500 ريال في الإمارات؟ ("How much is customs on a handbag worth 500 riyals in the UAE?") the gold call is `calculate_customs` with `category` = حقيبة يد, `destination_country` = الإمارات and `product_value` = 500.0. The gold call omits `currency` although a currency word is present, a per-tool convention we recovered from the training split.

Data and metrics. We used only the released splits: 10,550 training rows and 545 development rows, of which 500 require a call and 45 are negative. Twenty tools appear in development gold; the other seven occur only as distractors. Among the 500 call-required development rows the dialects are MSA 289, Gulf 72, Levantine 67, Egyptian 66 and Maghrebi 6. Overall A weights FnAcc/ArgEM $0.40/0.60$; Overall B weights FnAcc/ArgEM/ThinkRate $0.30/0.50/0.20$; ArgEM requires all key-value pairs to match under the official normaliser. We entered Tracks A and B; Track C is a dialect breakdown of the same predictions.

Relation to prior work. Tool use is normally learned: models are trained to emit calls (Schick et al., 2023) or tuned against API registries (Patil et al., 2024), with structural validity imposed at decode time by grammar-constrained decoding (Geng et al., 2023). The task baseline takes the first route, a LoRA-tuned Gemma 3 (270M) (Nacar et al., 2026b). Ours is orthogonal: same supervision source, but the learned decoder is replaced by retrieval plus gated extraction, so structural validity holds by construction rather than by constraint. Predictions trace to a span in the utterance, a convention mined from train, or a retrieved neighbour's value; only the first two are textual or canonical evidence, and not every emission passes the shipped gate (Appendix B).

3 System Overview

NABIQ is a chain of pure functions over the tuple $(tool, arguments, user_text, developer_context)$; no stage reads a row identifier or gold.

P1: Arabic normalisation. A normalised projection unifies Alef and Hamza variants, final Ya / Alef-maqsura and Ta-marbuta, strips diacritics and tatweel, and maps Eastern-Arabic digits (٠١٢٣٤٥٦٧٨٩) to Western. This absorbs orthographic and dialectal variation (جدة/جده, الاسبوع/الأسبوع) with no learned representation. Matching uses this view; emitted values are verbatim spans.

P2: Routing and the negative class. Character n-gram TF-IDF features (Spärck Jones, 1972; Pedregosa et al., 2011), computed within word boundaries, are fitted on the 10,550 training utterances. A multi-class logistic regression ranks tools and we take its highest-probability label that appears in the row's candidate set. A separate binary detector handles the negative class, where routing actually fails: past-completion statements (قارنت أسعار الجوالات وقررت آخذ سامسونج, "I compared phone prices and decided on Samsung"), capability questions and out-of-scope requests. It raised development FnAcc from 0.9706 to 1.0000. Its feature text also concatenates `negative_category`, which §5 shows to be a label proxy. The concatenation sits in the feature builder rather than in any decision rule, and §5 measures what it contributes.

P3: Argument extraction. Per tool we mine the training split for the expected key set and the canonical form of each value: ISO-3 currency codes, ISO-2 language codes, English enums such as `resignation`/`end_of_contract`, ISO dates, and Arabic-verbatim country and category names. Values come from a per-tool TF-IDF cosine nearest-argument retriever, which supplies the key set and a starting value from the most similar training utterance; from direct span extraction (regexes plus closed train-mined gazetteers for cities, specialties, procedures, months and number words); and from canonical maps.

P4: Guards. A *schema guard* drops keys absent from the tool schema and coerces types. A *per-tool evidence guard* then removes optional arguments whose support is missing from the text: it

covers the five tools where speculative filling cost us most, dropping `guests` without a quantity word, `doctor_name` without a title, and so on. It is not general, and Limitations says what that leaves open.

P5: Repair layers and acceptance. Extraction alone reached ArgEM ≈ 0.53. The rest of the development gain came from successive generations of narrow repair rules, each simulated *independently* on all 500 positive cases and admitted only on a win with *exactly zero* regressions. Twenty-one rules were accepted, 13 in the first repair stage and 8 in the second. Further candidates were rejected under the same test; our records do not support an exact rejected total, and Appendix D lists the main ones with win and regression counts.

P6: The evidence gate. A non-oracle selector runs after the repair layers. A candidate value replaces the base prediction only if it is (i) a normalised substring of the user text, (ii) a member of a closed canonical map (specialty, procedure, currency code, or one of ten transliterated names themselves present in the text), (iii) an ISO date whose day and month tokens both occur in the text, or (iv) a *removal* of an unsupported argument. Numerics require digits or Arabic number words.

P7: Track B. The trace comes deterministically from the Track A prediction via a parameterised Arabic template: one sentence naming the tool and enumerating the extracted `key=value` pairs, or a fixed out-of-scope sentence for `none`. Track B reuses Track A's tool and arguments byte-for-byte and adds only `think`, so reasoning-action consistency holds by construction; ThinkRate was 1.0000 on every run.

4 Experimental Setup

Train and development splits only: no external corpora, no pretrained model, no fine-tuning, no GPU. All development figures use the 545-row development split as released for the final development phase, and all training uses the 10,550-row training split from the same release. L1 figures are the official leaderboard's; L2 and L3 are scored by our own re-implementation of the metric, which we label v1.2 throughout and distinguish from the official scorer. Development gold was used solely for offline scoring and the zero-regression diagnostics; it is never read at prediction time, and the blind runner refuses any input path whose filename contains `gold`, `answer` or `label`. The environment is Python 3.13.13 with `scikit-learn` 1.9.0, `numpy` 2.5.1, `scipy` 1.18.0 and `pandas` 3.0.5, run locally on CPU; pins ship with the reproducibility package (Appendix A). Our local scorer reimplements the metric with a v1.2 "fair" normalisation (digit systems, orthographic variants, currency and language aliases, order-independent lists); on generations 12–13 the official scorer was 0.000–0.004 more lenient.

Determinism. Post-submission forensic testing identified a Python hash-order dependency in tied candidate selection: tie-breaking iterated unordered containers and so inherited the interpreter's hash order, so two runs in fresh processes could differ on a few rows. A pipeline with "no random seed" is not automatically deterministic; unordered iteration is a silent seed. Rerunning the archived package on the development-as-blind input under two different hash seeds changes 5 of 545 rows, all of them gazetteer tie-breaks between orthographic variants of the same city; repeating a run at a fixed seed is identical. We therefore report the archived package as reproducible *only* under a pinned `PYTHONHASHSEED`, and we do not claim that the submitted fallback was byte-identical across fresh processes. Sorting the affected traversals is the fix this diagnosis implies; it is not present in the archived package (Appendix A).

5 Results and Analysis

Levels of number. Table 1 keeps them apart: **L1** the official development leaderboard, **L2** a local ablation on the 500 positive development rows, **L3** the acceptance run on a development-as-blind input, **L3b** a rehearsal on held-out training rows, **L4** the blind test.

Official results. On the $1{,}125$-row blind test NABIQ obtained FnAcc $1028/1125 = 0.9138$, ArgEM 0.7081 and ThinkRate 1.0000, giving Overall A 0.7904 (rank 16) and Overall B 0.8282 (rank 15). Both official submission files, Track A and Track B, were produced by the fallback runner `run_nabiq_blind.py`; the primary `v15rx` runner was prepared but not submitted (see below). Track B reuses Track A's predictions unchanged. The winner scored Overall B 0.9400, and 0.087 of that 0.112 gap is ArgEM. We do not attach a significance claim to the ranking: separating neighbour-

Configuration	FnAcc	ArgEM	Ovr. A
L1. Official development leaderboard			
v8	1.0000	0.7820	0.8692
v13	1.0000	0.8860	0.9316
L2. Local ablation (500 pos., v1.2 scorer)			
v13 base	1.0000	0.8820	0.9292
+stage1 (repairs)	1.0000	0.9140	0.9484
+stage2 (toolpacks)	1.0000	**0.9380**	0.9628
L3. Acceptance run (dev-as-blind, 545 rows)			
v15rx primary	1.0000	0.9380	0.9628
fallback (submitted)	1.0000	0.9200	0.9520
L3b. Pseudo-blind rehearsal (545 held-out rows)			
v14 chain, seed 13	0.9963	0.6476	0.7871
L4. *Official blind test* (1,125 rows)			
fallback, Track A	0.9138	0.7081	**0.7904**

Table 1: Levels of evaluation. L2 and L3 reuse the 500 development cases the rules were selected on. L3b is a rehearsal on data the system never saw, run before the blind release; it predicts L4's *Overall A* to 0.0033, while the development estimate is off by 0.1724. This agreement is composite: L3b overestimates FnAcc by 0.0825 and underestimates ArgEM by 0.0605, and the 0.40/0.60 weighting cancels most of both (see text). A further L2 stage scored 0.9360 and was not kept; L3 strict ArgEM is 0.9160 / 0.8960.

System	FnAcc	ArgEM	Ovr. A	Ovr. B
GPT-4o, zero-shot	0.9324	**0.7442**	**0.8195**	—
270M fine-tuned	**0.9813**	0.6790	0.8000	0.8074
GPT-4o, 3-shot	0.8676	0.6991	0.7665	—
NABIQ (ours)	0.9138	0.7081	0.7904	**0.8282**
Random selection	0.0418	0.0060	0.0203	0.0155

Table 2: Blind test, re-scored by the organisers with the final argument normaliser (Nacar et al., 2026a). The pre-normaliser figures circulated during the campaign are not comparable and are not used here.

ing systems evaluated on the same instances would require a paired test, which we did not run.

Comparison with the baselines. Table 2 places NABIQ against the organisers' baselines, all re-scored with the released argument normaliser on the final gold so the figures are comparable. With no neural component NABIQ is on a par with the fine-tuned 270M model and few-shot GPT-4o on ArgEM and Overall A, below zero-shot GPT-4o, and ahead of the fine-tuned baseline on Overall B. The organisers' table does not report a ThinkRate column, so we compare Overall B directly and do not infer one.

What transferred. Argument extraction. The parts responsible are grounded in *train*: normalisation, train-mined canonical forms and key-set conventions, and the evidence gate, whose criterion is a property of the language, not of a 500-example sample. Generations 1–6, before any development-derived rule, already reached ArgEM 0.756 on development.

What did not. Function selection. FnAcc fell from 1.0000 on development to 0.9138 on test, below the function-selection accuracy of the published baselines in Table 2. The no-call detector's feature string concatenates `negative_category`, which is null for all 10,500 positive training rows and populated for all 50 negative ones, and which the organisers have confirmed is *nulled in the blind test*: the detector saw the label on both splits it was developed against and nothing at test time. Five-fold cross-validation on train, at the shipped threshold, gives no-call recall 0.9600 with the field and 0.5400 without it. Rerunning the whole frozen chain with the field blanked in train and input costs 0.0294 FnAcc and 0.0177 Overall A on development. That magnitude is about a third of the development-to-test FnAcc difference and about a tenth of our 0.1724 development-estimate error, but it is a development counterfactual and the remainder cannot be attributed to any single cause (Appendix C).

Evidence-gate conformance. Under the P6 criterion, which admits a numeric supported by digits *or* an Arabic number word, 907 of 988 emitted development values conform (91.8%); a surface-only diagnostic that credits digits but not number words gives 862 (87.2%). Of the 81 non-conforming values, 48 are relative dates resolved to ISO and 21 are retrieval-introduced strings absent from the utterance (2.1% of all values). Those 21 are what the gate does not reach: it governs candidate *replacements* and a subset of tools, so a first-pass retrieved value can survive without textual support (Appendix B).

The gap, measured in advance. The submitted fallback fell from ArgEM 0.9200 on the development-as-blind acceptance run (L3) to 0.7081 on test, a like-for-like drop of 0.2119; the primary v15rx configuration reached 0.9380 on development but was not the system evaluated at test. This pattern is consistent with adaptive development-set selection pressure, and the evidence predates the blind release. Two weeks out we carved 545 rows from train, stripped their gold, refitted every component on the remaining

10,005 rows and ran the frozen runner (L3b). It scored Overall A 0.7871 against an official 0.7904, an error of 0.0033, versus 0.1724 for our development estimate and 0.1412 for the last official leaderboard entry. This composite agreement is partly cancellation: L3b overestimated FnAcc (0.9963 against 0.9138, $+0.0825$) and underestimated ArgEM (0.6476 against 0.7081, -0.0605), and the $0.40/0.60$ weighting nets these to -0.0033, so it bounds the aggregate, not each component. It still anticipated the direction and scale of a drop the development leaderboard did not: four further seeds put held-out ArgEM between 0.6155 and 0.6685. The acceptance protocol looked strict but was applied 21 times to the *same* 500 examples: generations 1–6 encode train conventions, generations 7–14 encode development conventions, and only the first kind survived.

Acceptance coverage and the validator. For the blind run we prepared v15RX, the frozen chain plus two guarded rules, one re-extracting `recipient_iban` after an explicit account marker (the G2 rule). A post-run handoff record, written two days before the phase closed, states that the primary outputs were rejected because of invalid `recipient_iban` values and that the fallback outputs were the files uploaded for both tracks, having passed the validator with no errors or warnings. That record is our evidence for the rejection; the primary blind outputs themselves were not retained, so we could not re-run the validator on them to reconstruct the failure independently.

Two engineering failures interacted. The first is a contract mismatch: G2's token pattern admits a purely numeric token of six or more digits, whereas the validator requires an IBAN-like value, so the rule could emit values the contract rejects. The second is a path-coverage gap in acceptance, which measured the run (row count, validator verdict, Track A/B agreement) but not whether each new rule was exercised. Rerunning G2 over the 545-row development-as-blind acceptance input produces *zero* insertions across the 24 predicted `transfer_money` rows, so the acceptance suite never executed the G2 insertion path and the contract mismatch stayed hidden until the blind run. Neither failure alone would have been sufficient: a rule whose output the validator accepts would have survived poor coverage, and full coverage would have surfaced the mismatch. A useful acceptance criterion is per-rule: each new rule should fire on $\geq k$ rows with every emitted value checked against the validator contract. The configuration change cost 0.9380 vs. 0.9200 ArgEM at L3.

A label proxy in the input. The rows carry `negative_category`, which reaches the no-call detector's feature string. Appendix C measures its contribution and what follows for benchmark design.

Dialect robustness (Track C). Official per-dialect Overall: MSA 0.797, Gulf 0.798, Egyptian 0.798, Levantine 0.791, Maghrebi 0.647, a macro-average of 0.7661 and a gap of 0.151. Four dialects sit within 0.007; the entire gap is Maghrebi. Our L2 gap of 0.074 had suggested that surface spans confer dialect robustness, but Maghrebi is six development rows there, and the gazetteers come from a distribution in which it is 1.3% of the data.

Error analysis. Before the final generation 59 development cases were wrong, dominated by date (27) and span (20) errors; stage 2 fixed 28 with zero regressions. Most survivors coincide with a date-format variation we measured in the development gold for `search_hotels`, which alternates between ISO, verbatim Arabic and `DD-MM` for equivalent expressions (Appendix D). We report this as an observation about those development rows and about our own residual errors, not as a claim about the benchmark's ceiling or about the task overall.

6 Conclusion

NABIQ reaches ArgEM 0.7081 and Overall B 0.8282 with no LLM and no GPU, and the evidence gate is the transferable idea: where it applies, a candidate value must satisfy explicit textual or canonical evidence criteria. It does not apply to every emitted value, and Appendix B bounds the shortfall. The methodological result is cheaper than any architecture: for NABIQ, a rehearsal on held-out training rows produced a substantially closer estimate of the final aggregate than the repeatedly tuned development estimate (0.0033 against 0.1724 on Overall A), even though its FnAcc and ArgEM estimates were each off by more. This is a single-system result; it suggests that held-out rehearsals are worth running in similar shared-task settings, where a development split doubles as the tuning split.

Limitations and Future Work

The rule-selection protocol would be better served by a held-out split. Reserving 150 development cases and requiring every candidate rule to be non-negative there, as well as zero-regression on the remaining 350, trades live-leaderboard position for a less biased estimate, and the trade is the incentive such a board creates. The next system should be hybrid: a small fine-tuned model as the proposal distribution (Nacar et al., 2026b), especially for the call/no-call decision, with the evidence gate kept as the acceptance test: the shape AISA's governance layers (Nacar et al., 2026c) would ask for regardless.

The evidence gate is narrower in our code than the principle it implements. It governs candidate *replacements*, and the per-tool guard covers five of the twenty tools that appear in gold, so a value supplied by the nearest-argument retriever from a different training row can survive into a prediction without occurring in the user's text. The hallucination-resistance claim should therefore be read as scoped to the layers described in P4 and P6 rather than to every emitted value. Appendix B quantifies the shortfall on development: 2.1% of emitted values are strings absent from the utterance. Two limitations attach to that figure. It is development-only, because blind gold is not released, so we cannot report the same quantity for the test set. And it is a surface diagnostic rather than a measurement of the shipped gate, since it does not credit numerics supported by Arabic number words. Extending the gate to cover every first-pass emission, and measuring conformance directly against the shipped criterion, both remain future work.

As submitted, our extractors are per-tool and closed: adding a tool requires writing gazetteers and span rules for it, so the approach does not scale to an open registry the way a fine-tuned decoder does. The no-call detector is sensitive to unseen phrasing once the `negative_category` signal is absent, so its development figures reflect that signal as well as the model. All L2 and L3 numbers are computed on the same 500 development cases used for rule selection and are therefore optimistic. Our held-out estimates come from rehearsals carved out of train (L3b), which share train's gold conventions and contain almost no negative rows, so they bound argument extraction well and say nothing about no-call detection. We report no multiple-run variance on the blind test, and that absence is a limit of our evidence rather than a property of the system: the official score comes from the single submitted fallback output for each track, blind gold was never available to us for repeated evaluation, our repeated-run diagnostics use held-out-train rehearsal seeds rather than the blind split, and the archived code can itself vary across fresh processes through the documented hash-order dependency. The blind Track C breakdown shows the cost concretely: Maghrebi Overall is 0.647 against ≈ 0.79 for the other four dialects, and closing that gap needs training signal the released split barely contains.

Ethical Considerations

We collected no new data. All work uses the organisers' released splits. Two licences apply and we keep them distinct. Our own code is released under the MIT licence. The one data artefact we redistribute is a processed form of the organisers' training split, which is a derivative of the AISA-ArabicFC dataset and remains governed by that dataset's upstream licence, Apache 2.0, with attribution; we do not relicense it.

The corpus contains person names and bank-account-like strings as argument values for tools such as `transfer_money`. We treated these as task data and made no attempt to link them to real individuals; readers reusing the release should note that argument values, not only utterances, can carry person-like strings.

Dialect representation is the clearest fairness concern our numbers speak to. NABIQ performed substantially worse on the Maghrebi blind subset than on the other four dialect subsets: Overall 0.647 against ≈ 0.79. The development set contains only six call-required Maghrebi rows, and Maghrebi is 1.3% of the corpus, so our development-side evidence for this dialect is thin and the blind subset is the stronger signal. We read this as a representation concern about a system whose gazetteers and conventions are mined from this distribution, and as a reason not to report a single aggregate score alone. We do not extrapolate from these benchmark subsets to how the system would serve Maghrebi speakers in deployment.

The tool registry spans banking, government and health services, so an argument-level error is not cosmetic: a wrong `recipient_iban`

or `iqama` number has consequences outside the benchmark. The evidence gate reduces one class of that risk by refusing to emit values absent from the user's text, but it does not address the other direction, and our blind FnAcc of 0.9138 means the system still calls a tool on some utterances that ask for none. Neither this system nor its successor should be deployed on such tools without a confirmation step under human control.

Acknowledgments

We thank Tuwaiq Academy and the AISA-ArabicFC organizers for providing the dataset, the baseline model, and the evaluation framework for this shared task, and for their responsiveness throughout the evaluation period. We also thank the anonymous reviewers for their comments.

References

Ahmed Abdelali, Kareem Darwish, Nadir Durrani, and Hamdy Mubarak. 2016. Farasa: A fast and furious segmenter for Arabic. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Demonstrations*, pages 11–16.

Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. 2023. Grammar-constrained decoding for structured NLP tasks without fine-tuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 10932–10952. Association for Computational Linguistics.

Omer Nacar, Mohammed Al Khalifa, and Saeed Alzaharani. 2026a. AISA-ArabicFC: The first shared task on arabic function calling for agentic ai systems. In *Proceedings of The Fourth Arabic Natural Language Processing Conference: Shared Tasks*, Budapest, Hungary. Association for Computational Linguistics.

Omer Nacar, Deema Alquffari, Saleh Alsharideh, Adeem AlOtaibi, Abdulaziz Alabdulkarim, Leen Alhazmi, Nada Alomar, Wareef Alzubaidi, Nada Alsultan, Ahmed Alrabghi, et al. 2026b. From language to action in arabic: Reliable structured tool calling via data-centric fine-tuning. *arXiv preprint arXiv:2603.16901*.

Omer Nacar, A. Deema, and A. Mohammed. 2026c. AISA: A unified architecture for agentic ai systems.

Ossama Obeid, Nasser Zalmout, Salam Khalifa, Dima Taji, Mai Oudah, Bashar Alhafni, Go Inoue, Fadhl Eryani, Alexander Erdmann, and Nizar Habash. 2020. CAMeL tools: An open source python toolkit for arabic natural language processing. In *Proceedings of the Twelfth Language Resources and Evaluation Conference (LREC)*, pages 7022–7032.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems (NeurIPS)*.

F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Karen Spärck Jones. 1972. A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1):11–21.

A Configuration and Reproducibility

Nothing in NABIQ is trained by gradient descent, so there is no learning rate, batch size or epoch count. The complete set of fitted-model settings is:

- **Router.** `TfidfVectorizer` with `analyzer="char_wb"`, `ngram_range=(3,5)`, `min_df=2`, `max_features=120000`, feeding `LogisticRegression` with `solver="lbfgs"`, `max_iter=2000`, `class_weight= "balanced"`. Input text concatenates the user utterance, the dialect tag, and the name, description and parameter names of every candidate tool.

- **No-call detector.** Same vectoriser and classifier settings. Decision threshold 0.60 on the predicted probability of the call class. Its input text additionally concatenates `negative_category` (Appendix C).
- **Per-tool argument retriever.** One `TfidfVectorizer` per tool with `analyzer="char_wb"`, `ngram_range=(2,5)`, `min_df=1`, `max_features=80000`; cosine similarity, top-1 neighbour.

Tokenisation. We use no morphological segmenter: neither Farasa (Abdelali et al., 2016) nor CAMeL Tools (Obeid et al., 2020), and no model tokeniser. Character n-grams are computed directly on the normalised string by `scikit-learn`, which is why the P1 normalisation carries the load that a segmenter would carry elsewhere. We do not handle Arabizi or Latin-script code-switching; neither appears in the released splits.

Hardware and runtime. The system was executed locally on CPU, with no GPU and no external inference API. We did not preserve a run log recording core count or wall-clock time, so we make no exact historical runtime claim.

Which figures come from which run. The official leaderboard result corresponds to the single submitted fallback output for each track: Track A and Track B were both submitted from the fallback runner. A primary v15RX candidate was also generated locally on the blind input, but it was not submitted after failing validation, so the blind input was executed locally more than once even though only one output per track was scored officially. We did not receive repeated official evaluations. Development and development-as-blind figures come from reruns of the archived package, which reproduce at a fixed hash seed. The held-out rehearsal figures come from five pseudo-blind seeds used as a diagnostic; these are rehearsals on data carved from train, not repeated blind evaluations.

Package and determinism. The submission package contains the two approved runners, the mandatory validator, the pinned environment and the released training file used for routing, retrieval and convention mining. It contains no gold, answer or label file of any kind, and no notebooks or caches. Predictions are produced by `python src/run_nabiq_blind.py --input data/blind/test.jsonl --track-a A.jsonl --track-b B.jsonl`, followed by `src/validate_submission_files.py`, which checks row count, identifier uniqueness, field types, absence of `think` in Track A, and agreement between Tracks A and B on `id`, `tool_called` and `arguments`. The package is MD5-frozen and contains no sampling and no explicit seed. Its one source of run-to-run variation is the hash-order dependency described in Section 5: ties in the gazetteer-mining and candidate-collection paths are broken by iteration order over unordered containers. Measured on the development-as-blind input, two runs under different values of `PYTHONHASHSEED` differ on 5 of 545 rows, while two runs at the same value are identical. Reproducing our numbers therefore requires pinning `PYTHONHASHSEED`; we distinguish this archived package from the configuration that produced the official submission, which carried the same dependency and for which we make no byte-determinism claim.

B Direct Surface Grounding (Diagnostic)

This appendix reports a diagnostic, not a measurement of the shipped evidence gate. The two differ deliberately, and we state the difference rather than reconcile it by adjusting the number.

We measure on the 545 development rows using the frozen v14 Track A submission (`nabiq_v14_pc.jsonl`, MD5 `fa764e6d…`), so the figure is reproducible from that artifact. The measurement script accompanies this paper's supplementary package. Blind gold is not released, so this is a development-only measurement.

We report two quantities. **P6 conformance** applies the shipped criterion: a value conforms if, after the P1 normalisation, it is a substring of the user utterance, a member of one of the closed canonical maps (currency, language, Zakat type, termination type, Quran search type, country), an ISO date whose day and month tokens both appear, or a numeric supported by digits *or by an Arabic number word*. The **surface diagnostic** is the same test with the number-word clause removed, and is therefore a strict lower bound.

Of 988 emitted argument values, 907 conform to P6 (91.8%) and 862 (87.2%) are surface-grounded. The surface figure decomposes as 480 by substring, 142 by a canonical map, 234 by digits and 6 by date tokens; the further 45 that P6 admits are quantities

Tool	n	ArgEM	Tool	n	ArgEM
translate_text	26	1.000	convert_currency	24	0.917
check_iqama	25	1.000	order_food	24	0.917
check_traffic	24	1.000	check_insurance	27	0.889
check_visa	24	1.000	calculate_zakat	24	0.875
search_umrah	23	1.000	transfer_money	24	0.875
end_of_service	23	1.000	book_doctor	30	0.833
search_quran	24	0.958	compare_prices	24	0.833
get_air_quality	23	0.957	calculate_customs	24	0.833
search_medications	27	0.926	get_weather	29	0.828
get_qibla	25	0.920	search_hotels	26	0.731

Table 3: Per-tool ArgEM, development, v14, strict scoring; FnAcc is 1.000 for every tool, and the 45 none rows are all routed correctly. Tool names are abbreviated. search_hotels is weakest, matching the date-format inconsistency reported below.

written as Arabic cardinals, including dual forms such as شخصين for two and multiplier phrases such as نص مليون for 500,000.

The 81 values that do not conform to P6 are: 48 relative dates resolved to ISO from the developer context, 21 strings absent from the utterance (2.1% of all values), 8 quantities derived from a unit word rather than a number word (اسبوع yielding days = 7, which we do not count as a number word), and 4 numerics with no recoverable support.

That last group is the one the gate does not reach. As shipped it governs candidate *replacements* and a subset of tools, so a first-pass value taken from the nearest-argument retriever can survive without direct support; 2.1% bounds how often that happens on this development set. Extending the gate to every first-pass emission remains future work.

C The negative_category Leak

Each distributed row carries a negative_category field. In the released training split it is null for all 10,500 positive rows and populated for all 50 negative ones, with five values (general_knowledge, chitchat, opinion, ambiguous_underspecified, off_domain). It is therefore a perfect proxy for the call/no-call label. The organisers have since confirmed that the field is non-null exactly when requires_function is false across all 10,550 training rows and all 545 development rows, and that *it is nulled in the blind test*.

That asymmetry is the mechanism behind our largest development-to-test movement. A detector that consumes the field sees the label on both splits it was developed against and sees nothing at test time, so its development behaviour cannot transfer. We quantify the size of the effect below; it is smaller end to end than the detector-level comparison implies, because later rule-based no-call layers absorb much of the loss.

Our build_none_text helper concatenates this field into the string that the no-call detector vectorises, at training and at inference alike. We did not intend this and did not catch it before the freeze. The router, the nearest-argument retriever, the span extractors and both guards do not read the field; the no-call detector does.

To measure the effect we reran the shipped detector configuration post-submission, at the shipped threshold of 0.60, once with the field in the feature string and once with it blanked. Under five-fold cross-validation on train, no-call recall is 0.9600 (48/50) with the field and 0.5400 (27/50) without. Trained on train and evaluated on development, the same comparison gives 0.6222 against 0.5333.

The two numbers differ for an instructive reason. Train uses five negative categories; development uses eleven, six of which (past_tense_completed, feelings_experiences, meta_questions, about_domain_no_action, comparative_evaluation, hypothetical_conditional) never appear in train, so the memorised strings transfer to only about half of the development negatives. The leak therefore inflates the detector without being the whole explanation for development FnAcc of 1.0000, which the full chain reaches with later no-call layers as well. These are post-submission diagnostics and change no submitted prediction.

End-to-end ablation. The measurements above isolate the detector. To size the field's effect on the delivered system we reran the entire frozen chain twice on a development-as-blind input, once as shipped and once with negative_category

blanked in the training split *and* in the input, scoring both with our v1.2 scorer. The as-shipped arm reproduces the reported L3 fallback row exactly (FnAcc 1.0000, ArgEM 0.9200, Overall A 0.9520), which validates the replication.

	FnAcc	ArgEM	Ovr. A
As shipped	1.0000	0.9200	0.9520
Field blanked	0.9706	0.9100	0.9343
Difference	−0.0294	−0.0100	−0.0177

Table 4: End-to-end effect of blanking `negative_category` in both the training split and the input, on the frozen chain, development-as-blind, v1.2 scorer.

No-call recall over the 45 development negatives falls from $45/45$ to $35/45$; 16 rows change their tool decision and 6 positives are wrongly routed to `none`. The end-to-end cost is therefore much smaller than the detector-level comparison suggests, because later rule-based no-call layers recover most of what the detector loses.

This is a development-as-blind counterfactual, not an intervention on the blind test, so it sizes the field's contribution under that counterfactual rather than isolating a cause of the observed test result. Its magnitude, 0.0294 FnAcc, is roughly one third of the 0.0862 development-to-test FnAcc difference, and its 0.0177 on Overall A is roughly one tenth of the 0.1724 development-estimate error. The remaining difference cannot be uniquely attributed to a single cause; it is consistent with repeated development-set selection pressure, with the configuration difference between the tuned primary and the submitted fallback, and with distributional differences between the development and blind splits, and we do not attempt to apportion it among them.

The rerun used the shipped `scikit-learn` version, 1.9.0, under Python 3.11.15 rather than the originally recorded Python 3.13.13 environment. The as-shipped arm reproduces the recorded L3 fallback v1.2 figures exactly. Strict ArgEM differs from the archived 0.8960 by two rows; we cannot isolate whether that difference arises from the substitute runner's documented replay path, from the Python-version difference, or from their interaction, and we do not attribute it to any one of them.

Two recommendations follow for benchmark design. Removing `negative_category` from future releases, in train as well as in the blind file, closes the shortcut: the no-call decision carries 0.40 of the Track A composite, which is a large share to leave recoverable from a metadata field. More generally, a field can enter a system through a feature builder rather than through a decision rule, so an audit that inspects only the decision logic will not find it.

D Development Error Analysis

Per tool. Per-tool ArgEM for the v14 chain on the 545 development rows under our strict scorer, which gives overall FnAcc 1.0000 and ArgEM 0.9120. These are L2-level numbers on the split the rules were selected on, so read them as a map of where the extractors are weak, not as generalisation estimates. The blind gold is not released, so a per-tool breakdown at L4 is unavailable to us.

By error type. The 59 remaining development errors before the final generation, by type (a case may fall in several categories): wrong date 27, wrong span 20, wrong entity boundary 12, missing argument 8, extra argument 7, wrong number 4, wrong list 3. By argument key they concentrate in `check_in` and `check_out` (22 combined), followed by `specialty`, `medication_name` and `category`. Stage 2 of the final generation fixed 28 of the 59 with zero regressions.

Rejected rules. The main rejected candidates, with their (wins, regressions) on the 500 positive development cases: strip the definite article from `medication_name` (1, 11); `search_quran search_type=verse` for آية (1, 14); `calculate_zakat` type removal (1, 22); `get_weather days=1` for اليوم (4, 5); unconditional عملية strip from `procedure` (3, 8); Arabic-to-English name transliteration at large (2, 6); بعد غد canonicalisation (1, 1); and customs category Arabic-to-English maps, rejected on a training-majority check before simulation. A further candidate layer (the "G3" rule and a deletion gate) was rejected before the freeze and is absent from the submitted package.